

Questions on Unit 04 Sections 01-04

The following multiple-choice questions are designed to assess your knowledge of this unit's section. See the version without solutions first. Each question is worth 1.0 points, with the score being divided among the correct answers where applicable. You should aim to complete each question within an average time of three to four minutes. You may solve the questions in any order you prefer; once finished, compare your responses with the provided solution file. You have successfully passed the assessment if you achieve at least 50% of the available points.

Ex. Text and binary files

Choose one or more options

- ☐ In Unix/Linux, the kernel natively distinguishes between text and binary files using different file types.
- ☒ A file is basically a sequence of bytes written one after the other on disk.
- ☒ Each byte consists of 8 bits that can each be 0 or 1.
- ☒ We commonly distinguish between text files (ASCII or Unicode) and binary files, even if both are just bytes.
- ☐ A C++ source file is always treated as a binary file by user programs and cannot be read as text.

Ex. Newline conventions in text files

Choose one or more options

- ☒ On Unix/Linux and macOS, a newline is represented by a single character, typically Line Feed (LF, 10).
- ☒ On Windows, a newline is represented by a sequence of two characters: LF and CR together.
- ☐ Text files on Unix use Carriage Return followed by Line Feed as the newline sequence.
- ☒ Text files are usually “line-oriented”, with content logically grouped into lines separated by newline characters.
- ☐ The physical newline representation is always the same on all operating systems.
- .

Ex. Basic C++ I/O headers

Choose one or more options

- ☒ `<iostream>` provides the basic I/O functionalities for standard streams like `cin` and `cout`.
- ☒ `<sstream>` provides classes to perform input and output operations on `std::string` objects.
- ☒ `<fstream>` provides classes for file I/O operations on named files.
- ☐ The `<fstream>` header only allows binary file operations and cannot be used for text files.
- ☐ The `<sstream>` header is required to use `std::cin` and `std::cout`.

Ex. Stream states and error handling

Choose one or more options

- ☒ Once an input error occurs on a stream, subsequent I/O operations will generally fail until the state is cleared.
- ☒ Using `if (cin >> s) { ... }` is a common way to test if an input operation succeeded.
- ☐ `s.good()` returns false if the last I/O operation succeeded.
- ☒ `s.fail()` is true if an I/O operation failed (for example, reading a string into an integer).
- ☒ `s.clear()` resets all condition flags of a stream to a valid state.
- ☐ The idiom `while (!s.eof()) { ... }` is recommended as a robust input loop.

Ex. File modes with fstreams

Choose one or more options

- ☒ `std::ios::in` opens a file for input operations.
- ☒ `std::ios::out` opens a file for output operations.

- ☒ `std::ios::app` appends all writes at the end of the file, seeking to the end before each write.
- ☒ `std::ios::trunc` truncates the file, discarding its previous content on open (when used with output).
- ☐ `std::ios::binary` forces text mode, translating newlines when reading and writing
- ☒ `std::ios::ate` seeks to the end immediately after opening, but then allows repositioning with seek operations.

Ex. Opening files with ofstream

Choose one or more options

- ☒ `std::ofstream out("myfile");` opens myfile for output, truncating it by default.
- ☒ `std::ofstream out("myfile", std::ofstream::app);` opens myfile for appending, preserving existing content.
- ☐ `std::ofstream out("myfile", std::ofstream::in);` is the usual way to open a file only for reading.
- ☒ Modes can be combined with |, e.g. `std::ofstream::out | std::ofstream::trunc`.
- ☐ Truncation must always be specified explicitly; it is never the default.

Ex. Basic binary I/O. Given the following code snippet:

```
int v1 = 12345, v2;

std::ofstream outF("data.bin", std::ios::out | std::ios::binary);
if (outF.is_open()) {
    outF.write(reinterpret_cast<const char*>(&v1), sizeof(v1));
    outF.close();
}

std::ifstream inF("data.bin", std::ios::in | std::ios::binary);
if (inF.is_open()) {
    inF.read(reinterpret_cast<char*>(&v2), sizeof(v2));
    inF.close();
}
```

Choose one or more options

- ☒ `std::ios::binary` tells the library to treat the file as binary, avoiding newline translations.
- ☒ `reinterpret_cast<const char*>(&v1)` converts the address of v1 to a const char* pointer required by write.
- ☒ If the platform representation is unchanged, after a successful read we typically have `v1 == v2`.
- ☐ The code writes the ASCII characters '1', '2', '3', '4', '5' to the file.
- ☐ Using write and read is restricted to text files and cannot be combined with `std::ios::binary`.

Ex. Seeking in files

Choose one or more options

- ☒ `seekg` is used to move the next read position (get pointer) within a stream.
- ☒ `seekp` is used to move the next write position (put pointer) within a stream.
- ☒ `ios_base::beg`, `ios_base::cur`, and `ios_base::end` specify the reference position for the offset.
- ☐ The offset parameter in `seekg` and `seekp` is always interpreted in units of whole records, not bytes.
- ☐ The get and put pointers are always independent; using `seekg` never affects any write position.

Ex. Simple seek example. Given the following code snippet:

```
std::fstream file("example.bin", std::ios::binary | std::ios::out | std::ios::trunc);

file.write(reinterpret_cast<const char*>(&data1), sizeof(data1));
file.seekp(sizeof(data2), std::ios::cur);
file.write(reinterpret_cast<const char*>(&data3), sizeof(data3));
```

```
file.seekp(sizeof(data1), std::ios::beg);
file.write(reinterpret_cast<const char*>(&data2), sizeof(data2));
```

Choose one or more options

- ☒ The first write stores data1 at the beginning of the file.
- ☒ After seekp(sizeof(data2), std::ios::cur), the file position is advanced as if data2 had been written.
- ☒ The second seekp moves the position back to overwrite the reserved space for data2.
- ☐ The final layout in the file is data3, then data2, then data1, in that order.
- ☒ This pattern can be used to write records out of logical order while controlling their physical positions.

Ex. Using setw, left, right, internal. Given the following code snippet:

```
std::cout << std::right << std::setw(10) << "Value" << std::endl;
std::cout << std::right << std::setw(10) << 123 << std::endl;
std::cout << std::left << std::setw(10) << "Hello" << std::endl;
std::cout << std::internal << std::setw(10) << -456 << std::endl;
```

Choose one or more options

- ☒ std::setw(10) sets the width of the next output field to 10 characters.
- ☒ std::right causes the following text/numbers to be right-aligned in that field.
- ☒ std::left causes "Hello" to be left-aligned, padding to the right.
- ☒ std::internal aligns the sign separately from the magnitude for numeric output.
- ☐ std::setw permanently changes the width for all subsequent outputs until reset.

Ex. C-style integer arrays. Given the following code snippet:

```
const unsigned N1 = 10;
constexpr unsigned N2 = 3;
int v1[N1];
int v2[] = {1, 2, 3, 4, 5};
int v3[N1] = {1, 2, 3, 4, 5};
int v4[N2] = {1, 2, 3, 4, 5};
int v5[5] = {1, 2, 3, 4, 5};
v2 = v3;
```

Choose one or more options

- ☒ v1 is an uninitialized array of 10 integers.
- ☒ v2 is an array whose size is deduced from the initializer list {1, 2, 3, 4, 5}.
- ☒ v3 has 10 elements; the first 5 are initialized, the rest are value-initialized (typically 0).
- ☐ v4 is well-formed even if N2 is 3 and there are 5 initializers.
- ☒ v5 is a 5-element array fully initialized by the list.
- ☐ The assignment v2 = v3; is valid and copies the elements of v3 into v2.

Ex. C-style character arrays and C-strings. Given the following code snippet:

```
char s1[] = {'C', '+', '+'};
char s2[] = {'C', '+', '+', '\0'};
char s3[] = "C++";
char s4[3] = "C++";
```

Choose one or more options

- ☒ s1 is not null-terminated and therefore is not a proper C-style string.
- ☒ s2 explicitly contains the null terminator '\0'.
- ☒ s3 is equivalent to {'C', '+', '+', '\0'} and is a valid C-string.
- ☐ s4 is correct and stores "C++" including the terminating '\0' in three characters.
- ☒ Library functions that treat char* as strings rely on '\0' to find the end.

Ex. C-strings and common pitfalls. Given the following code snippet:

```
char s1[] = {'C', '+', '+'};
std::cout << s1;

char s2[] = "C--";
char s3[] = "C++";

s2 = s3;
if (s2 == s3) { /* ... */ }
```

Choose one or more options

- ☐ `std::cout << s1;` safely prints "C++" because the array has 3 characters.
- ☒ `std::cout << s2;` is valid since `s2` is null-terminated.
- ☒ The assignment `s2 = s3;` is illegal because arrays are not assignable.
- ☒ The expression `s2 == s3` compares addresses, not the contents of the strings.
- ☒ To copy C-style strings one must use library functions such as `strcpy`.

Ex. Pointers and arrays. Given the following code snippet:

```
int v[10];
int *p, *b, *e;

p = &v[0];
b = &v[0];
e = &v[10];

for (p = b; p < e; p++)
    std::cout << *p << std::endl;

p = v;
int *p2 = p + 4;
```

Choose one or more options

- ☒ `p = &v[0];` makes `p` point to the first element of the array.
- ☒ `b` points to `v[0]` and `e` points one past the last element `v[9]`.
- ☒ The loop prints all the elements from `v[0]` to `v[9]`.
- ☒ `p = v;` is equivalent to `p = &v[0];`.
- ☐ `p2 = p + 4;` always causes undefined behavior because pointer arithmetic is not allowed on arrays.

Ex. Multidimensional arrays and storage order. Given the following code snippet:

```
int m1[3][4];
int m2[3][4] = {
    {0, 1, 2, 3},
    {4, 5, 6, 7},
    {8, 9, 10, 11}
};
int m3[3][4] = {
    0, 1, 2, 3,
    4, 5, 6, 7,
    8, 9, 10, 11
};
```

Choose one or more options

- ☒ `m1` is an uninitialized 2D array of 3×4 integers.
- ☒ `m2` is initialized row by row with nested braces.
- ☒ `m3` uses a flat initializer list that is equivalent to `m2` due to row-major storage.
- ☐ C++ stores multidimensional arrays in column-major order by default.
- ☒ In C/C++, a “multidimensional array” is implemented as an array of arrays.

Ex. Range-for on multidimensional arrays. Given the following code snippet:

```
constexpr int R = 3;
constexpr int C = 4;
int m[R][C];

for (auto &r : m) {
    for (auto &c : r) {
        std::cin >> c;
    }
}
```

Choose one or more options

- ☒ The outer loop iterates over each row (which is itself an array of `int`).
- ☒ Using `auto &r` avoids copying entire rows and is necessary when working with a C-style array here.
- ☒ The inner loop with `auto &c` iterates over individual elements and allows their modification.
- ☐ Replacing `auto &r` with `auto r` would be equally correct and efficient in this case.
- ☒ Range-for with references is useful when avoiding “massive” data copies.

Ex. Buggy range-for nested loops (fixing code). Given the following code snippet::

```
for (auto r : m) {
    for (auto c : r) {
        std::cout << c;
    }
}
```

Choose one or more options

- ☐ The code is correct and equivalent to using references in both loops.
- ☒ The outer loop should use a reference (`auto &r`) to avoid copying rows.
- ☒ In the buggy slide, `r` is not a reference and thus `c` cannot iterate correctly over an element.
- ☒ A correct version to print elements is:

```
for (auto &r : m)
    for (auto c : r)
        std::cout << c;
```
- ☒ When we want to modify elements inside the inner loop, we must use `auto &c.`

Ex. Sequential containers – general properties

Choose one or more options

- ☒ A container is an object that stores other objects and manages the storage space for them.
- ☒ Sequential containers provide fast sequential access to their elements.
- ☒ With the exception of `std::array`, sequential containers support dynamic growth and shrinkage.
- ☐ All sequential containers guarantee constant-time insertion and removal at any position.
- ☒ Different sequential containers provide different performance characteristics for insertion, deletion, and random access.

Ex. Iterator categories in the standard library

Choose one or more options

- ☒ `std::vector` provides random-access iterators.
- ☒ `std::deque` provides random-access iterators.
- ☒ `std::list` provides bidirectional iterators.
- ☒ `std::forward_list` provides forward iterators.
- ☐ `std::array` provides only input iterators.
- ☒ `std::string` typically provides random-access iterators.

Ex. Basic vector operations

Choose one or more options

- ☒ `std::vector<T> v;` default-constructs an empty vector.
- ☒ `v.empty()` returns `true` if the vector has no elements.
- ☒ `v.size()` returns the number of elements stored in the vector.
- ☒ `v.push_back(a)` appends `a` at the end of the vector.
- ☐ `v[n]` performs bounds checking and throws an exception if `n` is out of range.
- ☒ Assigning `v1 = v2`; replaces the elements of `v1` with a copy of those in `v2`.

Ex. Vector initialization forms

Choose one or more options

- ☒ `std::vector<int> v1;` creates an empty vector of `int`.
- ☒ `std::vector<int> v2 = v1;` copies all elements from `v1` into `v2`.
- ☐ `std::vector<int> v{10};` always creates a vector of 10 elements all equal to 0.
- ☒ `std::vector<int> v(10, 1);` creates a vector with 10 elements, each equal to 1.
- ☒ List-initialization such as `std::vector<int> v{1,2,3};` explicitly specifies the element values.

Ex. Time complexity of vector operations

Choose one or more options

- ☒ Random access using `v[n]` is typically $O(1)$.
- ☒ Back insertion (`push_back`) is typically $O(1)$ but can be $O(n)$ in the worst case due to reallocation.
- ☒ Inserting or removing an element in the middle of a vector is $O(n)$.
- ☐ `std::vector` guarantees that `push_back` is always $O(1)$.
- ☒ Vectors often allocate capacity beyond the current size to amortize reallocation costs.

Ex. Iterator operations on vectors

Choose one or more options

- ☒ `auto b = v.begin();` yields an iterator to the first element.
- ☒ `auto e = v.end();` yields an iterator one past the last element.
- ☒ `*b` refers to the element denoted by iterator `b`.
- ☒ `b->mem` is equivalent to `(*b).mem`.
- ☐ Comparing iterators from different containers of the same type is always well-defined.

Ex. Iterators and vectors of strings (insertion). Given the following code snippet:

```
std::vector<std::string> v = {"one", "two", "three", "four"};
for (auto it = v.begin(); it != v.end(); ++it) {
    if (it->size() == 3) {
        it = v.insert(it, "foo");
        ++it;
    }
}
```

Choose one or more options

- ☒ The code scans the vector and inserts "foo" before each 3-letter string.
- ☒ After `insert`, the iterator `it` is updated to point to the newly inserted element.
- ☒ Incrementing `it` after insertion allows the loop to continue from the element following "foo".
- ☐ The insertion cannot invalidate any iterators, so updating `it` is unnecessary.
- ☒ At the end of the first loop, some original elements will be shifted to higher indices.

Ex. Iterators and vectors of strings (erasure). Given the following code snippet:

```
for (auto it = v.begin(); it != v.end(); ++it) {
    if (it->size() == 3) {
        it = v.erase(it);
    }
}
```

```

    } else {
        ++it;
    }
}

```

Choose one or more options

- ☒ `erase` removes the element pointed to by `it` and returns a new iterator to the next element.
- ☒ Assigning `it = v.erase(it);` is the correct way to keep the iterator valid after erasure.
- ☐ After `erase`, it is mandatory to increment `it` manually in addition to the returned value.
- ☒ Removing elements inside a loop must be done carefully to avoid using invalid iterators.
- ☐ `erase` never invalidates any iterator except the one passed to it.

Ex. Warning on iterator invalidation. Given the following code snippet:

```

std::vector<int> v = {0,1,2,3,4,5,6,7,8,9};
auto it = v.begin();
while (it != v.end()) {
    if (*it % 2) {
        it = v.insert(it, *it);
        it += 2;
    } else {
        it = v.erase(it);
    }
}

```

Choose one or more options

- ☒ Inserting into a `std::vector` can invalidate existing iterators.
- ☒ The code adjusts `it` after insertion and erasure to avoid skipping or reprocessing elements.
- ☐ `v.insert` and `v.erase` on vectors never invalidate iterators.
- ☒ After `erase`, the returned iterator points to the element that followed the erased one.
- ☒ This pattern demonstrates careful management of iterators when modifying a container inside a loop.

Ex. Associative containers – types

Choose one or more options

- ☒ `std::map` is an associative array holding key–value pairs.
- ☒ `std::set` is a container where each element is a key.
- ☒ `std::multimap` is a map allowing multiple elements with the same key.
- ☒ `std::multiset` is a set that can store multiple copies of the same key.
- ☒ `std::unordered_map` and `std::unordered_set` are organized using hash functions.
- ☐ `unordered_multimap` and `unordered_multiset` store their elements in sorted order.

Ex. Main operations on associative containers

Choose one or more options

- ☒ `c.insert(v)` inserts an element `v` into the associative container `c`.
- ☒ `c.emplace(args)` constructs an element from `args` and inserts it into `c`.
- ☒ `c.erase(k)` removes every element whose key is `k`.
- ☒ `c.erase(b, e)` removes all elements in the iterator range `[b, e)`.
- ☒ `c[k]` for a map returns the element with key `k`, inserting a default-initialized value if `k` is not present.
- ☐ `c.at(k)` silently inserts a new element if the key `k` is not found.

Ex. Extra operations on associative containers

Choose one or more options

- ☒ `c.find(k)` returns an iterator to the first element with key `k` or `c.end()` if not found.

- ☒ `c.count(k)` returns the number of elements with key `k`.
- ☒ `c.lower_bound(k)` returns an iterator to the first element with key not less than `k`.
- ☒ `c.upper_bound(k)` returns an iterator to the first element with key greater than `k`.
- ☒ `c.equal_range(k)` returns a pair of iterators delimiting the range of elements with key `k`.
- ☐ `c.count(k)` always returns either 0 or 1, even in a multimap.

Ex. Maps and the pair type

Choose one or more options

- ☒ Elements of `std::map` are of type `std::pair<const Key, T>`.
- ☒ Each pair has public data members `first` and `second`.
- ☒ `make_pair(v1, v2)` creates a `std::pair` initialized with `v1` and `v2`.
- ☐ The members of `std::pair` are private and accessible only via getters.
- ☒ Two pairs `p1` and `p2` are equal if both `p1.first == p2.first` and `p1.second == p2.second`.

Ex. Word frequency map – simple version. Given the following code snippet:

```
std::map<std::string, std::size_t> word_count;
std::string word;
```

```
while (std::cin >> word)
    ++word_count[word];

for (const auto &w : word_count) {
    std::cout << w.first << " occurs "
               << w.second << " time(s).\n";
}
```

Choose one or more options

- ☒ `word_count` maps each distinct word to its frequency.
- ☒ `++word_count[word];` inserts a new key with value 0 if the word is not present, then increments it.
- ☒ The range-for loop visits elements in key order (for `std::map`).
- ☐ Using operator[] on a `std::map` does not allow inserting new keys.
- ☒ This pattern is a canonical example of counting word frequencies with a map.

Ex. Word frequency using insert and pair. Given the following code snippet:

```
while (std::cin >> word) {
    auto ret = word_count.insert({word, 1});
    if (!ret.second)
        ++ret.first->second;
}
```

Choose one or more options

- ☒ `insert({word, 1})` tries to insert the pair `<word, 1>` into the map.
- ☒ `ret.second` is true if a new element was inserted and false if the key already existed.
- ☒ `ret.first` is an iterator to the element with the given key, regardless of insertion success.
- ☐ If the word already exists, `insert` overwrites its previous count with 1.
- ☒ When insertion fails, incrementing `ret.first->second` updates the existing count.

Ex. Sets and uniqueness. Given the following code snippet:

```
std::set<int> is = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
auto it1 = is.find(1);
auto it2 = is.find(11);
auto n1 = is.count(1);
auto n2 = is.count(11);
```

Choose one or more options

- ☒ `is` is a set containing integers from 0 to 9.
- ☒ `it1` refers to the element with key 1.
- ☒ `it2` equals `is.end()` because 11 is not in the set.
- ☒ `n1` is 1 because key 1 appears exactly once.
- ☒ `n2` is 0 because key 11 is absent.
- ☐ A `std::set` can contain multiple elements with the same key.

Ex. Building a set from a vector. Given the following code snippet:

```
std::set<int> myset;
std::vector<int> iv = {2,4,6,8,2,4,6,8,2,4,6,8};
myset.insert(iv.begin(), iv.end());
myset.insert({1,3,5,7,1,3,5,7});
```

Choose one or more options

- ☒ After the first `insert`, `myset` contains {2,4,6,8}.
- ☒ After the second `insert`, `myset` contains {1,2,3,4,5,6,7,8}.
- ☐ The duplicates cause `insert` to throw an exception.
- ☒ `insert` with iterator range will ignore duplicate keys in a `std::set`.
- ☒ List initialization can be passed directly to `insert` for multiple values.

Ex. Using a set to exclude words. Given the following code snippet:

```
std::map<std::string, std::size_t> word_count;
std::set<std::string> exclude = {
    "The", "But", "And", "Or", "An", "A",
    "the", "but", "and", "or", "an", "a"
};
std::string word;

while (std::cin >> word)
    if (exclude.find(word) == exclude.end())
        ++word_count[word];
```

Choose one or more options

- ☒ `exclude` stores words to ignore when counting frequencies.
- ☒ The condition `exclude.find(word) == exclude.end()` means “word is not in the exclude set”.
- ☒ An alternative is to check `exclude.count(word) == 0`.
- ☐ Using a `std::set` here prevents us from iterating over the excluded words.
- ☒ The map still counts only non-excluded words.

Ex. Generic algorithms and headers

Choose one or more options

- ☒ Generic algorithms are defined so they operate on elements of different types via iterators.
- ☒ The `<algorithm>` header contains many of the most important STL algorithms.
- ☒ Other algorithm-related headers include `<numeric>`, `<memory>`, and `<cstdlib>`.
- ☐ Every algorithm is a member function of a container class.[
- ☒ It is more important to understand the structure and usage pattern of algorithms than to memorize every variant.

Ex. Predicates in generic algorithms

Choose one or more options

- ☒ A predicate is an expression or function object that can be called and returns a value used as a condition.
- ☒ Algorithms often have versions that accept a user-defined predicate to customize behavior.
- ☒ For comparisons, default predicates usually correspond to operators like `<` or `==`.

- ☒ In the slides, `up` denotes a unary predicate and `bp` a binary predicate.
- ☐ Predicates used by STL algorithms must always return an `int`.

Ex. Search algorithms

Choose one or more options

- ☒ `find(b, e, v)` returns an iterator to the first element equal to `v` in `[b, e)`.
- ☒ `find_if(b, e, up)` returns an iterator to the first element for which `up` is true.
- ☒ `count(b, e, v)` counts how many elements are equal to `v`.
- ☒ `count_if(b, e, up)` counts how many elements satisfy the predicate `up`.
- ☐ `all_of(b, e, up)` returns an iterator to the last element that satisfies `up`.

Ex. Search algorithm example on a vector. Given the following code snippet:

```
std::vector<int> v = {2, 6, 1, 7, 3, 7};
auto res1 = std::find(v.begin(), v.end(), 7);
auto res2 = std::find(v.begin(), v.end(), 9);
if (res2 == v.end())
    std::cout << "Not found!";
```

Choose one or more options

- ☒ `res1` points to the first element equal to 7.
- ☒ `res2` is equal to `v.end()` because 9 is not present.
- ☒ The `if` condition correctly checks whether 9 is absent from the vector.
- ☐ Using `std::find` on an unsorted vector is undefined behavior.
- ☒ Search algorithms typically return `end()` when the search fails.

Ex. Binary search algorithms

Choose one or more options

- ☒ Binary search algorithms require the input range to be sorted.
- ☒ `lower_bound(b, e, v)` returns the first position where `v` could be inserted without breaking ordering (first not less than `v`).
- ☒ `upper_bound(b, e, v)` returns the first element greater than `v`.
- ☒ `equal_range(b, e, v)` returns a pair `{lower_bound, upper_bound}` for `v`.
- ☐ `binary_search(b, e, v)` returns an iterator to the found element or `end()` if not found.

Ex. Binary search example – lower_bound indices. Given the following code snippet:

```
std::vector<int> arr1 = {10, 15, 20, 25, 30, 35};
std::vector<int> arr2 = {10, 15, 20, 20, 25, 30, 35};
std::vector<int> arr3 = {10, 15, 25, 30, 35};
std::cout << lower_bound(arr1.begin(), arr1.end(), 20) - arr1.begin() << std::endl;
std::cout << lower_bound(arr2.begin(), arr2.end(), 20) - arr2.begin() << std::endl;
std::cout << lower_bound(arr3.begin(), arr3.end(), 20) - arr3.begin() << std::endl;
```

Choose one or more options

- ☒ For `arr1`, the printed index is 2.
- ☒ For `arr2`, the printed index is also 2 (first not less than 20).
- ☒ For `arr3`, the printed index is 2, corresponding to the first element greater than 20 (25).
- ☐ For `arr3`, `lower_bound` returns `arr3.end()` because 20 is absent.
- ☒ Subtracting iterators gives the zero-based index within the vector.

Ex. Binary search example – membership test. Given the following code snippet:

```
std::vector<int> arr = {10, 15, 20, 25, 30, 35};
if (std::binary_search(arr.begin(), arr.end(), 15))
    std::cout << "15 exists in vector";
else
    std::cout << "15 does not exist";
```

```

if (std::binary_search(arr.begin(), arr.end(), 23))
    std::cout << "23 exists in vector";
else
    std::cout << "23 does not exist";

```

Choose one or more options

- ☒ The first `binary_search` call returns `true` because 15 is in the vector.
- ☒ The second `binary_search` call returns `false` because 23 is not in the vector.
- ☐ `std::binary_search` modifies the container to insert missing values.
- ☒ `std::binary_search` returns a `bool` indicating whether the value exists in the sorted range.
- ☐ The vector must be sorted in descending order for `binary_search` to work.

Ex. Sorting algorithms

Choose one or more options

- ☒ `std::sort(b, e)` sorts the range `[b, e)` using a default comparison (`<`).
- ☒ `std::stable_sort(b, e, bp)` preserves the relative order of equivalent elements.
- ☒ `std::is_sorted(b, e, bp)` returns a `bool` indicating whether the range is sorted according to `bp`.
- ☒ `std::partial_sort(b, mid, e, bp)` places the smallest elements (according to `bp`) into `[b, mid)`.
- ☐ Sorting algorithms in `<algorithm>` can only be used with `std::vector<int>`.

Ex. Sorting integers with `std::sort`. Given the following code snippet:

```

std::vector<unsigned> v = {3,4,1,2};
std::sort(v.begin(), v.end());

```

Choose one or more options

- ☒ After the call to `std::sort`, `v` becomes `{1,2,3,4}`.
- ☒ By default, `std::sort` uses the less-than operator `<` for comparisons.
- ☐ `std::sort` cannot be used with unsigned integers.
- ☒ The call requires random-access iterators, which `std::vector` provides.
- ☐ The algorithm guarantees stability (preserves order of equal elements) by specification.

Ex. Sorting with custom predicate (strings). We want to sort strings by length:

```

bool isShorter(const std::string &s1, const std::string &s2) {
    return s1.size() < s2.size();
}

std::vector<std::string> words = { ... };
// BUGGY: missing predicate
std::sort(words.begin(), words.end());

```

Choose one or more options

- ☒ `isShorter` is a binary predicate comparing string lengths.
- ☒ To sort by length we should call `std::sort(words.begin(), words.end(), isShorter);`.
- ☐ The call `std::sort(words.begin(), words.end());` already uses `isShorter` automatically.
- ☒ Without specifying the predicate, `std::sort` orders strings lexicographically using `<`.
- ☒ We can also use a lambda instead of `isShorter`.

Ex. Sorting records with lambdas. Given the following code snippet:

```

struct Student {
    int id;
    std::string name;
    double grade;
};

void sortByID(std::vector<Student> &students) {
    std::sort(students.begin(), students.end(),
        [](const Student &a, const Student &b) {

```

```

        return a.id < b.id;
    });
}

```

Choose one or more options

- ☒ The lambda defines an ordering based on `id` in ascending order.
- ☒ `std::sort` uses the lambda to compare `Student` objects.
- ☐ Without a custom comparator, `std::sort` could still sort `Student` because it is a POD type.
- ☒ A similar lambda can sort by name using `a.name < b.name`.
- ☒ Another lambda can sort by grade descending using `a.grade > b.grade`.

Ex. Displaying records – code completion Given the following code snippet:

```

void displayRecords(const std::vector<Student>& students) {
    for (const auto &student : students) {
        // COMPLETE ME
    }
}

```

Which lines correctly complete the loop body?

Choose one or more options

- ☒ `std::cout << "ID: " << student.id << ", Name: " << student.name << ", Grade: " << student.grade << std::endl;`
- ☐ `cout << ID << name << grade << endl;` (missing qualifiers and text)
- ☒ We must qualify `std::cout` and `std::endl` or use `using` declarations.
- ☐ We must print only the `id`, as printing `name` and `grade` is not allowed with `std::cout`.

Ex. Buggy code snippet – using `ifstream`. Given the following code snippet:

```

std::string filename("input.txt");
int number;
std::ifstream infile(filename);
if (!infile.is_open()) {
    std::cerr << "Error: " << filename << std::endl;
    return EXIT_FAILURE;
}
// ... Process file ...
infile.close();

```

Choose one or more options

- ☒ `std::ifstream infile(filename);` opens the file `input.txt` for reading.
- ☒ Checking `!infile.is_open()` is a way to test if the file was successfully opened.
- ☒ `std::cerr` is appropriate for reporting error messages.
- ☐ We must call `infile.open(filename, std::ios::out)` instead to read the file.
- ☒ Calling `infile.close()` explicitly closes the file stream.

Ex. Buggy code – assignment of `ofstream`s. Given the following code snippet:

```

std::ofstream fo1, fo2;
...
fo2 = fo1;

```

Choose one or more options

- ☒ The code is ill-formed because I/O stream objects cannot be assigned.
- ☐ The assignment copies both the underlying file descriptor and the buffer contents.
- ☒ To reuse a stream variable, we should close it and open a new file instead of assigning.
- ☒ The slide explicitly notes “Error: cannot assign streams”.
- ☐ Assigning streams is allowed but only for `ifstream`, not for `ofstream`.

Ex. sstream library – typical uses

Choose one or more options

- ☒ `<sstream>` enables parsing data from strings using `std::istringstream`.
- ☒ `<sstream>` enables formatting data into strings using `std::ostringstream`.
- ☒ The library is useful for converting between strings and other data types.
- ☐ `<sstream>` only allows reading from files, not from `std::string`.
- ☒ Operations on string streams are syntactically similar to operations on `cin` and `cout`.